```
/*
 * April-2021
 *
 * Koristeći WCF kreirajte sistem za registraciju vozila.
 * Potrebno je da servis podržava sledeće funkcionalnosti:
 *
 * Registracija vozila (proslediti Vlasnika (ime, prezime i jmbg) i Vozilo (marka, model i boja)).
 * Vraća listu svih vozila zadatog vlasnika.
 * Vraća listu svih vlasnika zadatog modela.
 * Vraća listu svih vozila i njihovih vlasnika.
 */

namespace WCF.Contracts
{
    [ServiceContract]
    public interface IRegistration
    {
        [OperationContract]
        void RegisterVehicle(Owner owner, Vehicle vehicle);

        [OperationContract]
        List<Vehicle> GetVehiclesByOwnerJMBG(string jmbg);

        [OperationContract]
        List<Owner> GetOwnersByVehicleModel(string model);

        [OperationContract]
        List<Vehicle> GetAllVehicles();
    }

    [DataContract]
    public class Owner
    {
        [DataMember]
        public string FirstName { get; set; }

        [DataMember]
        public string LastName { get; set; }

        [DataMember]
        public string JMBG { get; set; }

        [DataMember]
        public List<Vehicle> Vehicles { get; set; }

        public Owner()
        {
            Vehicles = new List<Vehicle>();
        }
    }

    [DataContract]
    public class Vehicle
    {
        [DataMember]
        public string Brand { get; set; }

        [DataMember]
        public string Model { get; set; }

        [DataMember]
        public string Color { get; set; }

        [DataMember]
        public Owner Owner { get; set; }

        public Vehicle()
        {
            Owner = null;
        }
    }
}

namespace WCF.Services
{
    [ServiceBehavior(InstanceContextMode = InstanceContextMode.Single)]
    public class Registration : IRegistration
    {
        private readonly List<Vehicle> vehicles;
        private readonly List<Owner> owners;

        public Registration()
        {
            vehicles = new List<Vehicle>();
            owners = new List<Owner>();
        }

        public List<Vehicle> GetAllVehicles()
        {
            return vehicles;
        }

        public List<Owner> GetOwnersByVehicleModel(string model)
        {
            return vehicles.Where(v => v.Model == model && v.Owner != null)
                           .Select(v => v.Owner)
                           .Distinct()
                           .ToList();
        }

        public List<Vehicle> GetVehiclesByOwnerJMBG(string jmbg)
        {
            Owner owner = owners.Find(o => o.JMBG.Equals(jmbg));
            return owner?.Vehicles;
        }

        public void RegisterVehicle(Owner owner, Vehicle vehicle)
        {
            if (owner == null || vehicle == null)
                return;

            Owner existingOwner = owners.Find(o => o.JMBG == owner.JMBG);

            if (existingOwner == null)
            {
                existingOwner = owner;
                existingOwner.Vehicles = new List<Vehicle>();
                owners.Add(existingOwner);
            }
```

(IsReference = true) // mora da koristi u sebi referencirani tip Vehicle

(IsReference = true) // mora da koristi u sebi referencirani tip Owner

```
            existingOwner.Vehicles.Add(vehicle);
            vehicle.Owner = existingOwner;
            vehicles.Add(vehicle);
        }
    }
}
```

Add web.config
„basicHttpBinding"

```
namespace ClientConsoleApp
{
    internal class April2021
    {
        public April2021()
        {
            using (RegistrationClient registrationClient = new RegistrationClient())
            {
                while (true)
                {
                    Console.WriteLine("Enter the corresponding command:");
                    Console.WriteLine("1 - Register a vehicle");
                    Console.WriteLine("2 - Show vehicles owned by a person (by JMBG)");
                    Console.WriteLine("3 - Show owners of a specific vehicle model");
                    Console.WriteLine("4 - Show all vehicles and their owners");
                    Console.WriteLine("5 - Exit");

                    string command = Console.ReadLine().Trim();

                    switch (command)
                    {
                        case "1":
                            RegisterVehicle(registrationClient);
                            break;

                        case "2":
                            ShowVehiclesByOwner(registrationClient);
                            break;

                        case "3":
                            ShowOwnersByModel(registrationClient);
                            break;

                        case "4":
                            ShowAllVehiclesWithOwners(registrationClient);
                            break;

                        case "5":
                            return;

                        default:
                            Console.WriteLine("Unknown command.");
                            break;
                    }
                }
            }
        }

        private void RegisterVehicle(RegistrationClient proxy)
        {
            Console.WriteLine("Enter owner's first name:");
            string firstName = Console.ReadLine().Trim();
            Console.WriteLine("Enter owner's last name:");
            string lastName = Console.ReadLine().Trim();
            Console.WriteLine("Enter owner's JMBG:");
            string jmbg = Console.ReadLine().Trim();
            Console.WriteLine("Enter vehicle brand:");
            string brand = Console.ReadLine().Trim();
            Console.WriteLine("Enter vehicle model:");
            string model = Console.ReadLine().Trim();
            Console.WriteLine("Enter vehicle color:");
            string color = Console.ReadLine().Trim();

            var owner = new Owner()
            {
                FirstName = firstName,
                LastName = lastName,
                JMBG = jmbg,
            };
            var vehicle = new Vehicle()
            {
                Brand = brand,
                Model = model,
                Color = color,
            };
            proxy.RegisterVehicle(owner, vehicle);
        }

        private void ShowVehiclesByOwner(RegistrationClient proxy)
        {
            Console.WriteLine("Enter JMBG to show vehicles owned by a person:");
            string jmbg = Console.ReadLine().Trim();
            Vehicle[] vehicles = proxy.GetVehiclesByOwnerJMBG(jmbg);
            if (vehicles != null && vehicles.Length > 0)
            {
                foreach (Vehicle vehicle in vehicles)
                {
                    Console.WriteLine($"Brand: {vehicle.Brand}, Model: {vehicle.Model}, Color: {vehicle.Color}");
                }
            }
            else
            {
                Console.WriteLine("No vehicles found for the provided JMBG.");
            }
        }

        private void ShowOwnersByModel(RegistrationClient proxy)
        {
            Console.WriteLine("Enter vehicle model to show owners:");
            string model = Console.ReadLine().Trim();
            Owner[] owners = proxy.GetOwnersByVehicleModel(model);
            if (owners != null && owners.Length > 0)
            {
                foreach (Owner owner in owners)
                {
                    Console.WriteLine($"First Name: {owner.FirstName}, Last Name: {owner.LastName}, JMBG: {owner.JMBG}");
                }
            }
            else
            {
                Console.WriteLine("No owners found for the provided vehicle model.");
            }
        }
```

```
        private void ShowAllVehiclesWithOwners(RegistrationClient proxy)
        {
            Console.WriteLine("All vehicles and owners:");
            Vehicle[] vehicles = proxy.GetAllVehicles();
            if (vehicles != null && vehicles.Length > 0)
            {
                foreach (Vehicle vehicle in vehicles)
                {
                    Console.WriteLine($"Brand: {vehicle.Brand}, Model: {vehicle.Model}, Color: {vehicle.Color}, First Name: {vehicle.Owner.FirstName}, Last Name: {vehicle.Owner.LastNa
                }
            }
            else
            {
                Console.WriteLine("No vehicles found.");
            }
        }
    }
}

namespace ClientConsoleApp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            new April2021();
        }
    }
}
```